COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Hands on Activity: Arrays, Pointers and Dynamic Memory**

1. PROCEDURE OUTPUT

ILO A and B

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student (std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

    //deconstructor
    ~Student (){
        std::cout << "Destructor Called." << std::endl;
    }


    //Copy Constructor
    Student(const Stud (const char [24])"Copy Constructor Called"
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;

    }

    //Display Attributes
    void printDetailes(){
        std::cout << this->studentName << " " << this-> studentAge << std::endl;
    }
```

```
};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;
return 0;
}

```

Output:

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observations	I observed in this code that the name Roman will be added in Student 3 first before proceeding to student 1 and it will add its student age then it will copy the first constructor onto student 2 and then adding the student John Doe from the public constructor to the Student 3.

Table 2-2

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student (std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

    //deconstructor
    ~Student (){
        std::cout << "Destructor Called." << std::endl;
    }


    //Copy Constructor
    Student(const Stud (const char [24])"Copy Constructor Called"
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;

    }

    //Display Attributes
    void printDetailes(){
        std::cout << this->studentName << " " << this-> studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	I observed that the code works where the names are added on the studentList, namesList and ageList. And each time a name and age is added on each list a constructor will be called and once it was filled it then start to remove the list and then the destructor will be called removing the names and ages from the list.

Table

Loop A	

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student (std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

    //deconstructor
    ~Student (){
        std::cout << "Destructor Called." << std::endl;
    }


    //Copy Constructor
    Student(const Stud (const char [24])"Copy Constructor Called"
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;

    }

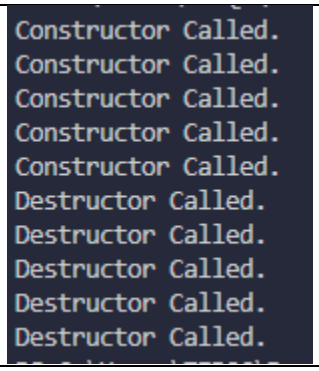
    //Display Attributes
    void printDetailes(){
        std::cout << this->studentName << " " << this-> studentAge << std::endl;
    }
```

```cpp
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] ={"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList [j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++) {//Loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

return 0;
}
```

	
Observation	The code is still the same however for the loop a added an array that will only view the first object student
Loop B	

```cpp
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] ={"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList [j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++) {//Loop B
       studentList[i].printDetails();
    }


 return 0;
}
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	In this code that I observed the code remains the same however in the last part instead of making an array it started to print the names and details of each students inside the array and it will print the number of times of the students meaning 5 will be printed out.

Output

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student (std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };

    //deconstructor
    ~Student (){
        std::cout << "Destructor Called." << std::endl;
    }


    //Copy Constructor
    Student(const Stud (const char [24])"Copy Constructor Called"
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;

    }

    //Display Attributes
    void printDetailes(){
        std::cout << this->studentName << " " << this-> studentAge << std::endl;
    }
```

```cpp
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] ={"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList [j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++) {//Loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

return 0;
}
```

```
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] ={"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList [j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++) {//Loop B
       studentList[i].printDetails();
    }


 return 0;
}
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
John Doe 18
John Doe 18
John Doe 18
John Doe 18
John Doe 18
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	The final output of the code remains the same as I said from the Loop A and B code however the difference is that the Loop A adds an array on the specific Student and Loop B prints 5 time on a specific student and its details.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C:
Code:

```cpp
#include <iostream>
#include <string>
#include <iomanip>

//Base Class
class GroceryItem {
protected :
    char* name;
    double price;
    int quantity;
    std::string category;

public:

    //Constructor
    GroceryItem(const char* item_name = "", double item_price = 0.0, int item_qty = 0, std::string item_cat = "General")
        : price(item_price), quantity(item_qty), category(item_cat) {

        int len = 0;
        while (item_name[len] != '\0') len++;

        name = new char[len + 1];
        for (int i = 0; i <= len; i++) {
            name[i] = item_name[i];
        }
    }

    //Destructor
    virtual ~GroceryItem() {
        delete[] name;
    }

```

```
33      //Copy Constructor
34      GroceryItem(const GroceryItem& other)
35          : price(other.price), quantity(other.quantity), category(other.category) {
36
37              int len = 0;
38              while (other.name[len] != '\0') len++;
39
40              name = new char[len + 1];
41              for (int i = 0; i <= len; i++) {
42                  name[i] = other.name[i];
43              }
44          }
45
46          //Copy Assignment Operator
47          GroceryItem& operator=(const GroceryItem& other) {
48              if (this != &other) {
49                  delete[] name;
50
51                  price = other.price;
52                  quantity = other.quantity;
53                  category = other.category;
54
55                  int len = 0;
56                  while (other.name[len] != '\0') len++;
57
58                  name = new char[len + 1];
59                  for (int i = 0; i <= len; i++) {
60                      name[i] = other.name[i];
61                  }
62              }
63              return *this;
64          }
65
```

```
66          //Function to calculate individual item total sum
67          double calculateSum() const {
68              return price * quantity;
69      }
70
71      //Displays the function to print the details
72      virtual void display() const {
73          std::cout << std::left << std::setw(12) << name
74                    <<std::setw(12) << category
75                    <<"PHP" << std::setw(8) << price
76                    <<std::setw(6) << ("x" + std::to_string(quantity))
77                    <<"PHP" << calculateSum() << std::endl;
78      }
79
80      const char* getName() const {return name;}
81  };
82
83  class Fruit : public GroceryItem {
84  public:
85  Fruit(const char* name = "", double price = 0.0, int quantity =0)
86      : GroceryItem(name, price, quantity, "Fruit") {}
87
88  };
89
90  class Vegetable : public GroceryItem {
91      public:
92          Vegetable(const char* name = "", double price = 0.0, int quantity = 0)
93              : GroceryItem(name, price, quantity, "Vegetable") {}
94
95  };
96
```

```cpp
double TotalSum(GroceryItem* list[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; ++i) {
        if (list[i] != nullptr) {
            total += list[i]->calculateSum();
        }
    }
    return total;
}

int main() {
    const int ITEM_COUNT = 4;

    GroceryItem* GroceryList[ITEM_COUNT];

    GroceryList[0] = new Fruit("Apple", 10.0, 7);
    GroceryList[1] = new Fruit("Banana", 10.0, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);


    //Displays the initial Grocery List
    std::cout << "Jenna's Grocery List is:" << std::endl;
    std::cout << std::left << std:: setw(12) <<"Item name"
            << std::setw(12) << "Category"
            << std::setw(12) << "Price"
            << std::setw(6) << "Qty"
            << "Total" << std::endl;


    for (int i = 0; i < ITEM_COUNT; ++i) {
        if (GroceryList[i] != nullptr) {
            GroceryList[i]->display();
        }
```

```
    }


    std::cout << "Total Cost For All Items is: " << TotalSum(GroceryList, ITEM_COUNT) << "\n\n";

    //Deletes Lettuce and de-allocates the dynamic memory
    std::cout << "Removing Lettuce form the List" << std:: endl;
    for (int i = 0; i < ITEM_COUNT; ++i) {
        if (GroceryList[i] != nullptr && std::string(GroceryList[i]->getName()) == "Lettuce") {
            delete GroceryList[i];
            GroceryList[i] = nullptr;
            std::cout << "Lettuce has been successfully removed from the List\n\n";
            break;
        }
    }

    //Displays the updated list and recalculates the total price
    std::cout << "The New Total Costs for All Items is:" << TotalSum(GroceryList, ITEM_COUNT) << std::endl;

    for (int i = 0; i < ITEM_COUNT; ++i) {
        delete GroceryList[i];
        GroceryList[i] = nullptr;
    }

    return 0;
}
```

Conclusion:
In this activity, I gained a comprehensive understanding about arrays, pointers and dynamic memory in C++ by creating the base class and the derived fruit and vegetables classes, I learned how to structure inheritance hierarchies and properly assign category types. Implementing the constructor, destructor, copy constructor and copy assignment operator to guarantee deep copies prevent memory leaks and eliminate dangling pointers. Overall I successfully met all the activity objectives, significantly improving my proficiency in memory safety with a key area for future improvement being the thorough testing of edge cases involving null pointer checks to build even more resilient software.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **To receive explanations of the required C++ code, programing concepts and proper syntax for the supplementary activity.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used solely as a learning and debugging reference tool to explain C++ syntax and other concepts and memory management rules. All class structures, logic implementation, testing and final code assembly were written, reviewed and finalized by me and AI did not replace my work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.